

Aula 2 — Desafios dos Dados N-Dimensionais

Da Ciência Tradicional ao Big Data
Novos Formatos e Fluxos para Dados Oceanográficos

Tobias Ferreira

Instituto Nacional de Pesquisas Oceânicas (INPO)

01/09/2026

Na aula anterior vimos **formatos, metadados e padrões**.

Agora vamos olhar para o problema que motiva grande parte dos fluxos cloud-native:

Pergunta central

O que acontece quando nossos dados N-dimensionais deixam de ter alguns megabytes e passam a ocupar gigabytes, terabytes ou petabytes?

Objetivos da aula

Ao final desta aula, você deverá conseguir:

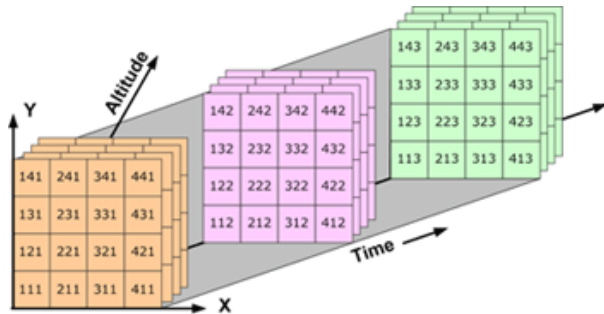
- descrever como NetCDF, GRIB e HDF5 organizam dados N-dimensionais;
- explicar por que os volumes de dados ambientais cresceram tanto nas últimas décadas;
- reconhecer limitações práticas ao abrir, mover e compartilhar arquivos grandes;
- explicar por que o acesso parcial é preferível ao download completo;
- relacionar chunking, memória, paralelismo e escalabilidade;
- usar xarray para inspecionar a estrutura de arquivos NetCDF e GRIB.

- Como NetCDF, GRIB e HDF5 representam grandes arrays?
- Por que os datasets cresceram tanto?
- O que acontece quando abrimos esses arquivos com xarray?
- Por que baixar o arquivo inteiro nem sempre faz sentido?
- Como chunking e acesso parcial ajudam?
- Onde formatos tradicionais começam a encontrar limites?

Dados ambientais são naturalmente N-dimensionais

Uma variável pode depender de várias dimensões:

- longitude;
- latitude;
- altura ou profundidade;
- tempo;
- membro de ensemble;
- tempo de previsão;
- variável ou cenário.



Exemplo

$$T = T(x, y, z, t, m)$$

Por que isso se torna difícil?

Mais dimensões

Mais combinações possíveis de índices e subconjuntos.

Mais resolução

Muito mais células para cada instante.

Mais tempo

Arquivos e coleções crescem continuamente.

O desafio não é apenas armazenar os dados, mas encontrá-los, movê-los e acessar somente o que precisamos.

NetCDF: arrays auto-descritivos

- Criado para dados científicos orientados a arrays.
- Armazena **variáveis**, **dimensões** e **atributos** no mesmo container.
- Permite que ferramentas descubram a estrutura do dataset durante a leitura.
- NetCDF-3 usa uma organização mais simples e contígua.
- NetCDF-4 utiliza HDF5 como camada de armazenamento e suporta grupos, grandes arrays e estruturas mais complexas.

Visão do usuário

Uma variável pode aparecer como `sst(time, latitude, longitude)` ou `temperature(time, depth, latitude, longitude)`.

GRIB: compacto e orientado à operação

- Formato da WMO para transmissão e armazenamento eficiente de campos meteorológicos em grade.
- Organizado como uma sequência de **mensagens**.
- Cada mensagem contém um campo e metadados codificados.
- Usa tabelas da WMO para parâmetros, níveis e outros descritores.
- GRIB2 ampliou metadados, compressão e representação de valores ausentes.

Vantagem e desafio

É excelente para fluxos operacionais, mas um arquivo heterogêneo pode precisar ser filtrado ou reorganizado antes de parecer um único array N-dimensional.

HDF5: container hierárquico de alto desempenho

- Formato geral para datasets grandes e complexos.
- Organiza dados em **grupos**, **datasets** e **atributos**.
- Muito usado em sensoriamento remoto e produtos de observação da Terra.
- É a camada de armazenamento utilizada pelo NetCDF-4.

Importante

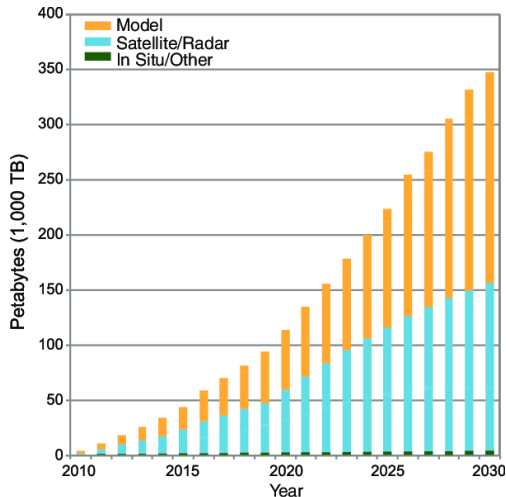
Quando usamos NetCDF-4, normalmente interagimos com o modelo NetCDF e não com os detalhes de baixo nível do HDF5.

Trinta anos de crescimento dos dados

Os volumes cresceram por causa de:

- maior resolução espacial;
- maior resolução temporal;
- mais níveis verticais;
- arquivos históricos mais longos;
- ensembles de previsão e clima.

Coleções que antes ocupavam MB hoje podem alcançar TB ou PB.



Baseado em Overpeck et al. (2011).

O problema em escala real

c			
Dataset	Área / aplicação	Nº de arquivos	Tamanho
CHES-SCAPE	Projeções climáticas britânicas a 1 km, 1980–2080	≈387.840	≈11 TB
NOC NEMO NPD	Simulações oceânicas globais NEMO em diferentes resoluções	não consolidado	≈71 TB
CMIP6 / ESGF	Modelos climáticos globais e experimentos acoplados	dezenas de milhões	≈21 PB
ERA5 – superfície	Reanálise atmosférica e de superfície do ECMWF	≈5,31 milhões	≈6 TB
ERA5 completo	Reanálise global horária	sem contagem única	ordem de

Mensagem

O desafio deixa de ser apenas “abrir um arquivo” e passa a envolver arquitetura de acesso, organização, computação e compartilhamento.

Um pequeno aumento pode virar um grande problema

Suponha uma grade com $N_x \times N_y$ pontos.

Se dobrarmos a resolução em cada direção horizontal:

$$(2N_x)(2N_y) = 4N_xN_y$$

Ou seja, temos aproximadamente **quatro vezes mais células** por instante.

Agora combine isso com:

- mais níveis de profundidade;
- saídas a cada 10 minutos;
- 30 anos de dados;
- dezenas de membros de ensemble.

Como trabalhamos com esses dados hoje?

Em muitos fluxos científicos:

- NetCDF é usado para análise e arquivamento;
- GRIB é usado para produtos operacionais e intercâmbio;
- xarray apresenta os dois como datasets N-dimensionais rotulados;
- arquivos ficam em discos locais, servidores compartilhados ou HPC;
- usuários baixam ou copiam dados antes de executar a análise.

Ponto importante

A interface de alto nível pode parecer simples, mas o desempenho depende da organização física, do tamanho dos arquivos, do hardware e do padrão de acesso.

Exercício 1 — Inspeccionando um NetCDF

Dataset: ERA5 SST, com campos 2D de temperatura da superfície do mar.

```
import xarray as xr

base_path = (
    "/gws/ssde/j25b/atlantis_vis/"
    "cloud-native-geoscience-course/"
)

ds = xr.open_dataset(
    f"{base_path}data/era5_sst/ocean_temperature.nc"
)

print(ds)
print(ds.dims)
print(ds.coords)
print(ds.attrs)
```

Exercício 1 — O que observar?

- Quais dimensões existem?
- Como tempo, latitude e longitude aparecem?
- Quais coordenadas estão associadas às dimensões?
- Que atributos descrevem a origem e as convenções do arquivo?
- Como a estrutura mudaria se adicionássemos profundidade ou ensemble?

Exercício 2 — Comparando NetCDF e GRIB

```
ds_nc = xr.open_dataset(  
    f"{base_path}data/era5_sst/ocean_temperature.nc"  
)  
  
ds_grib = xr.open_dataset(  
    f"{base_path}data/era5_sst/ocean_temperature.grib",  
    engine="cf_grib",  
)  
  
print(ds_nc)  
print(ds_grib)  
print(ds_grib.data_vars)
```

Pergunta: os dois formatos parecem muito diferentes quando vistos pelo xarray?

NetCDF e GRIB vistos pelo xarray

No exemplo da aula, ambos representam praticamente o mesmo campo científico:

- mesma grade espacial;
- 120 passos de tempo;
- uma variável principal: sst.

Mas existem diferenças:

- o NetCDF usa uma representação CF mais direta;
- o GRIB preserva coordenadas auxiliares e metadados específicos do formato;
- o GRIB é internamente baseado em mensagens, mesmo que o xarray apresente uma visão N-dimensional.

Exercício 3 — Pensando em escala

Imagine que:

- a resolução horizontal dobra em latitude e longitude;
- a frequência muda de 1 hora para 10 minutos;
- o período aumenta para 30 anos;
- vários grupos precisam analisar o mesmo dataset.

Discuta:

- 1 Como as dimensões e o número de células mudam?
- 2 Como o tamanho do arquivo e do arquivo histórico mudam?
- 3 O que acontece com memória, leitura e tempo de processamento?
- 4 Como você compartilharia esses dados?

O modelo tradicional: baixar primeiro

Publicar NetCDF/GRIB em portal, FTP ou HTTP



Baixar o arquivo completo



Salvar no computador ou servidor



Abrir e extrair o subconjunto necessário

Funciona bem para dados pequenos. Escala mal para grandes arquivos históricos.

Por que baixar tudo é ineficiente?

- O usuário pode querer apenas uma variável.
- Talvez precise de uma pequena região geográfica.
- Talvez precise de alguns dias de uma série de 30 anos.
- O download completo consome rede, tempo e armazenamento.
- Vários usuários criam cópias duplicadas do mesmo dataset.
- Cópias locais podem terminar em versões diferentes do mesmo dado.

Objetivo

Levar a computação até o dado, ou permitir que o usuário leia apenas os bytes necessários.

Acesso parcial e subsetting

Em vez de transferir o arquivo inteiro, queremos solicitar:

- uma caixa espacial;
- um intervalo de tempo;
- uma profundidade;
- uma variável;
- alguns membros de ensemble.

Protocolos como **OPeNDAP** e APIs modernas já permitem subsetting no servidor.

Limitação

Isso reduz transferência, mas exige infraestrutura de servidor e o desempenho ainda depende da carga do serviço e da rede.

Chunking: dividir para acessar melhor

Chunking divide um array grande em blocos menores.

Array monolítico

Ler uma pequena região pode envolver grandes operações de I/O.

Array dividido em chunks

Podemos buscar somente os blocos que intersectam o subconjunto desejado.

Chunking conecta organização física, memória, paralelismo e desempenho.

POSIX / filesystem tradicional

- arquivos e diretórios;
- disco local ou filesystem montado;
- comum em HPC e servidores;
- operações tradicionais de arquivo.

Object storage

- dados armazenados como objetos;
- acesso por APIs;
- escala horizontal;
- adequado para grandes arquivos históricos compartilhados;
- comum em infraestrutura cloud.

Problema de compatibilidade

NetCDF/HDF5 clássicos foram desenvolvidos principalmente para sistemas de arquivos POSIX, não para acesso direto e altamente paralelo a object storage.

Xarray é lazy, mas isso não resolve tudo

- Ao abrir um NetCDF, xarray pode inspecionar metadados sem carregar todos os valores.
- Isso é chamado de **lazy loading**.
- Porém, quando uma operação precisa dos valores, partes grandes do arquivo podem ter de ser lidas.
- Se o dataset não estiver organizado de acordo com o padrão de acesso, a leitura pode continuar cara.

Consequência

A interface Python pode ser simples, mas a organização dos dados continua determinando o custo real da análise.

Formato	Escala	Acesso
Como os arrays e metadados são organizados?	Quantos bytes, arquivos, dimensões e usuários existem?	Quais partes do dataset realmente precisamos ler?

Esses três fatores precisam ser considerados juntos.

Por que isso nos leva ao cloud-native?

- Grandes coleções não devem depender de downloads completos.
- Precisamos de acesso parcial eficiente.
- O armazenamento precisa escalar sem multiplicar cópias.
- O processamento precisa explorar paralelismo.
- A organização física deve combinar com o padrão de acesso.

Próximo passo

Nas próximas aulas veremos como formatos cloud-native, especialmente **Zarr**, combinam chunks, object storage e ferramentas como xarray e Dask para atacar esses problemas.

- Dados oceanográficos e meteorológicos são naturalmente N-dimensionais.
- NetCDF, GRIB e HDF5 continuam fundamentais para a ciência e a operação.
- Resolução, frequência, arquivos históricos e ensembles multiplicaram o volume dos dados.
- O fluxo “baixar tudo e analisar depois” deixa de escalar.
- Acesso parcial, chunking e processamento paralelo tornam-se essenciais.
- Formato, escala e padrão de acesso precisam ser pensados em conjunto.
- Esses desafios motivam os formatos e arquiteturas cloud-native.